Industrial Functional Programming [1]

Melinda Tóth, István Bozó

Dept. Programming Languages and Compilers
Eötvös Loránd University, Budapest, Hungary

[1] Supported by TÁMOP-4.1.2.A/1-11/1-2011-0052

Contents

OTP

- Open Telecom Platform
- Application & Libraries:
  - Basic (ERTS, stdlib, kernel, system architecture support library)
  - Database (Mnesia, ODBC)
  - Interface & Communication (Java/C interface, SSH, SSL, XML parsing, Inets)
  - Tool applications (Pman, TV, Debugger, Edoc, Syntax tools, Dialyzer)
- System design principles

OTP behaviours

- Design Patterns
- Framework with generic code (start, receive-send, terminate (crash))
- Separated generic and application specific parts
- Callback module – server – generic behaviour module

Example – client-server

Generic

- Server spawn
- Storing the ”Loop-Data“
- Sending request to the server
- Response to the client
- Receiving response from the server
- Stopping the server

Specific

- Initialisation of the server
- Loop-Data
- Client requests
- Handling the requests
- Responses from the server
- Termination, cleaning up

OTP behaviours

- Generic Servers
- Generic Finite State Machine
- Generic Event Handler/Manager
- Supervisors
- Applications

Pro & con

Pro

- Less code
- Less bugs
- Well tested framework
- Built-in functionality (log, trace, statistics)
- Expandable
- Common programing style
- Component based terminology

Con

- Extra effort to learn
- Affects the performance

Outline

Generic Servers

- client-server
- `gen_server.erl` contains the generic part
- call back module for the specific part
- `-behaviour(gen_server).`

Starting a Server

```
gen_server:start_link(SrvName, CBModName,
                      Args, Opts)
gen_server:start(SrvName, CBModName, Args, Opts)
gen_server:start_link(CBModName, Args, Opts)
gen_server:start(CBModName, Args, Opts)
```

- Server name: `{local, Name}`, `{global, Name}`
- Calls: `CBModName:init(Args)`,
  which returns: `{ok, LoopData}`

Passing Messages

```
gen_server:call(Name, Message)
gen_server:call(Name, Message, Timeout)
gen_server:cast(Name, Message)
gen_server:cast(Name, Message, Timeout)
```

- Call: Synchronous – Reply
- Cast: Asynchronous

```
handle_call(Message, From, LoopData) ->
  ...
  {reply, Reply, NewLoopData}.
handle_cast(Message, LoopData) ->
  ...
  {noreply, NewLoopData}.
```

Stopping the Server

```
handle_call(Message, From, LoopData) ->
   ...
   {stop, Reason, Reply, NewLoopData}.
handle_cast(Message, LoopData) ->
   ...
   {stop, Reason, NewLoopData}.
```

- `gen_server:call/cast*`
- Reason: `normal | shutdown | term() | {shutdown,term()}`
- Calls: `CBModName:terminate(Reason, LoopData)`

-behaviour(gen_server)

- Unexpected/system messages: `handle_info/2`
- e.g. `{'EXIT', Pid, Reason}`
- Hot code Upgrade: `code_change/3`

```
handle_info(Msg, State) ->
    ...
    {noreply, NewState}.
code_change(OldVsn, State, Extra) ->
    ...
    {ok, NewState}.
```

On the Next Lecture ...

- Supervisors
- Applications